

07. Tổng Hợp Chuyên Sâu - Nắm Library74 Để Có Thể Code Lại

Tài liệu này là phần tổng hợp sau khi đã học FE, DB, BE, AI, Deploy và Testing. Mục tiêu không phải chỉ biết dự án dùng công nghệ gì, mà là hiểu dự án được chia lớp như thế nào, dữ liệu đi qua đâu, mỗi module chịu trách nhiệm gì, và nếu phải code lại Library74 từ đầu thì nên bắt đầu từ đâu.

Tư duy học file này:

Không học thuộc file.
Không học thuộc tên class.
Học cách hệ thống được tách trách nhiệm.
Học luồng dữ liệu từ màn hình tới database, AI, mail và deploy.
Học cách tự dựng lại từng lát cắt chức năng.

Các file học trước:

- 1. [learn/01_FE.md](#) - Frontend.
- 2. [learn/02_DB.md](#) - Database.
- 3. [learn/03_BE.md](#) - Backend.
- 4. [learn/04_AI.md](#) - AI.
- 5. [learn/05_DEPLOY.md](#) - Deploy/DevOps.
- 6. [learn/06_TESTING.md](#) - Testing.

Trong file này ta ghép tất cả lại thành một bản đồ kỹ thuật của project.

1. Library74 Là Hệ Thống Gì?

Library74 là một hệ thống quản lý thư viện có AI.

Nó không chỉ là website giới thiệu sách. Nó có nhiều nhóm người dùng:

Nhóm	Vai trò
Public visitor	Xem trang public, tìm sách, xem thông tin thư viện
Student/User	Đăng nhập, tìm sách, đặt trước, xem sách đang mượn, phí phạt, ticket
Librarian	Quản lý sách, bản sao, mượn trả, yêu cầu, contact inbox
Admin	Quản lý người dùng, thủ thư, chính sách, audit log

Các mảng nghiệp vụ chính:

- Auth: đăng ký, đăng nhập, Google OAuth, refresh token, verify email, reset password.
- Catalog: publication, item/copy, author, publisher, category, tag, document.
- Circulation: đặt trước, mượn sách, trả sách, phí phạt, chính sách lưu thông.

- User: profile, notification, contact ticket, admin management.
- Recommendation: wishlist, rating, search history, semantic search, recommendation.
- AI: PDF ETL, embedding, vector search, recommendation engine, metadata.
- Deploy: Docker Compose, Caddy, frontend, backend, AI, database, queue.

Trả lời hội đồng:

Library74 là một hệ thống quản lý thư viện nhiều vai trò, có đầy đủ nghiệp vụ từ catalog, mượn trả, đặt trước, phí phạt đến notification, contact support và AI. Kiến trúc được tách thành frontend React, backend Spring Boot multi-module, AI FastAPI/worker và PostgreSQL/pgvector.

2. Kiến Trúc Tổng Thể

Nhìn từ ngoài vào trong:

```
Browser
-> Caddy reverse proxy
-> Frontend React hoặc Backend Spring Boot
-> PostgreSQL / Kafka / AI service / Mail
-> AI worker / RabbitMQ / pgvector
```

Ở production:

Service	Công nghệ	Vai trò
frontend	React + Vite + nginx	Giao diện người dùng
backend	Spring Boot	API và nghiệp vụ
ai-api	FastAPI	API AI cho search/recommendation/process
ai-worker	Celery	Xử lý PDF/vector/metadata nền
postgres	PostgreSQL + pgvector	Dữ liệu chính và vector
kafka	Kafka	Event/email/notification
rabbitmq	RabbitMQ	Queue tác vụ AI
caddy	Caddy	HTTPS và reverse proxy

Luồng request giao diện:

```
User mở https://library74.uk
-> Caddy
-> frontend:80
-> React render page
```

Luồng request API:

```
React gọi /api/publications
-> Caddy route /api/*
-> backend:8080
-> Controller
-> UseCase/Service
-> Repository
-> PostgreSQL
```

Luồng AI:

```
Backend gọi ai-api
-> FastAPI xử lý semantic search/recommendation
-> PostgreSQL pgvector hoặc recommender
```

Luồng xử lý PDF nền:

```
Backend có publication/document
-> gửi task
-> RabbitMQ
-> ai-worker
-> download PDF
-> extract text
-> chunk
-> embedding
-> lưu vector
-> callback backend bằng HMAC
```

Điểm cần nắm để code lại:

Project không phải một app monolith đơn giản. Nó là một hệ thống nhiều service, nhưng backend vẫn là trung tâm nghiệp vụ. Frontend không đi thẳng vào database hay AI worker. Mọi thao tác nghiệp vụ chính đi qua backend.

3. Frontend: Cách App Được Tổ Chức

Frontend nằm ở:

```
LMS_FE/
```

Các nhóm file chính:

```
App.tsx
contexts/
```

```
api/  
components/  
pages/  
utils/  
types.ts  
constants.ts
```

3.1. App.tsx Là Trục Điều Hướng

App.tsx định nghĩa:

- Router.
- Public routes.
- User routes.
- Librarian routes.
- Admin routes.
- Error routes.
- ProtectedRoute.
- Provider bọc toàn app.
- Lazy loading page.

Các prefix chính:

```
/publicpage  
/userpage  
/librarianpage  
/adminpage
```

Ví dụ:

```
/publicpage/search  
/userpage/dashboard  
/userpage/contact-tickets  
/librarianpage/contact-inbox  
/adminpage/users
```

3.2. Vì Sao Dùng HashRouter?

App dùng:

HashRouter

URL dạng:

```
https://library74.uk/#/userpage/contact-tickets
```

Lợi ích:

- Frontend route nằm sau #.
- Server chỉ cần serve `index.html`.
- Tránh lỗi refresh route trên static hosting nếu server chưa rewrite đầy đủ.

3.3. ProtectedRoute

`ProtectedRoute` kiểm tra:

```
Người dùng đã đăng nhập chưa?  
Role có đúng không?  
Nếu chưa login -> về login.  
Nếu sai role -> về dashboard phù hợp.
```

Ý tưởng:

```
student chỉ vào /userpage/*  
librarian chỉ vào /librarianpage/*  
admin chỉ vào /adminpage/*
```

3.4. Context Là Trạng Thái Toàn App

Frontend có các context:

```
AuthContext  
LanguageContext  
ThemeContext  
NotificationContext  
UploadContext  
AppDialogContext
```

Vai trò:

Context	Quản lý
<code>AuthContext</code>	token, userType, login/logout
<code>LanguageContext</code>	tiếng Việt/English
<code>ThemeContext</code>	light/dark/system
<code>NotificationContext</code>	notification realtime hoặc trạng thái thông báo

Context	Quản lý
<code>UploadContext</code>	upload PDF/cover/document
<code>AppDialogContext</code>	dialog dùng chung

Khi code lại, đừng nhét tất cả state vào từng page. State dùng chung phải đưa vào context hoặc store.

3.5. API Layer

API nằm ở:

```
LMS_FE/api/
```

Ví dụ:

```
authService.ts
publicationsService.ts
reservationService.ts
transactionsService.ts
fineService.ts
contactService.ts
recommendationService.ts
semanticSearchService.ts
adminService.ts
```

Ý tưởng:

```
Page không nên tự viết axios trực tiếp quá nhiều.
Page gọi service.
Service gọi axiosInstance.
axiosInstance xử lý token, language, refresh token, unwrap response.
```

3.6. `axiosInstance.ts` Là Cửa Ra API

`axiosInstance` làm các việc quan trọng:

- Set `baseUrl` từ `VITE_API_BASE_URL`.
- Set timeout từ `VITE_API_TIMEOUT`.
- Gửi `Authorization: Bearer <token>` nếu có.
- Gửi `Accept-Language` theo language đang chọn.
- Unwrap `response.data`.
- Xử lý token expired theo backend code `1403`.
- Dùng refresh token để lấy access token mới.
- Nếu refresh fail thì xóa token và redirect login.

Luồng:

```
Page -> service -> axiosInstance -> backend
```

Trả lời hội đồng:

Frontend được tách thành page, component, context, service API và utility. `App.tsx` quản lý routing và phân quyền theo route. `axiosInstance` là điểm tập trung để gắn token, ngôn ngữ và xử lý refresh token, giúp các page không lặp logic gọi API.

4. Backend: Multi-Module Theo Domain

Backend nằm ở:

```
LMS_BE/
```

Đây là Spring Boot multi-module Maven project.

Các module chính:

Module	Vai trò
<code>library-bootstrap</code>	App khởi động, config, migration, main application
<code>library-shared</code>	Thành phần dùng chung như email, audit, storage
<code>library-auth-module</code>	Auth, token, security, OAuth, reset password
<code>library-user-module</code>	User, role, notification, contact, admin management
<code>library-catalog-module</code>	Publication, item, author, publisher, category, tag
<code>library-circulation-module</code>	Borrow, return, reservation, fine, policy
<code>library-recommendation-module</code>	Wishlist, rating, search history, AI gateway

4.1. Vì Sao Tách Module?

Nếu tất cả code nằm trong một module duy nhất, project dễ rối.

Tách module giúp:

- Mỗi domain có ranh giới rõ.
- Dễ tìm code.
- Dễ test module riêng.
- Giảm phụ thuộc lộn xộn.
- Hội đồng nhìn thấy thiết kế có tổ chức.

4.2. Mô Hình Layer Trong Backend

Một module thường có các lớp:

```
presentation/controller  
application/usecase hoặc application  
domain/repository  
infrastructure/persistence  
infrastructure/config hoặc kafka
```

Ý nghĩa:

Layer	Vai trò
Presentation	Nhận HTTP request, validate, trả response
Application	Xử lý use case/nghiệp vụ
Domain	Interface repository, model nghiệp vụ
Infrastructure	JPA entity, repository implementation, external service

Luồng chuẩn:

```
Controller  
-> UseCase  
-> Domain Repository interface  
-> Infrastructure Repository implementation  
-> JPA / JDBC / Database
```

4.3. Vì Sao Có Interface UseCase?

Ví dụ:

```
CreateReservationUseCase  
CreateReservationUseCaseImpl
```

Lợi ích:

- Controller phụ thuộc vào hợp đồng, không phụ thuộc chi tiết.
- Dễ mock khi test controller.
- Dễ thay implementation nếu cần.
- Tách "làm gì" và "làm như thế nào".

4.4. Entity, DTO, Mapper

Backend thường phân biệt:

Entity: map với database.
DTO/Request/Response: dữ liệu vào/ra API.
Domain model: mô hình nghiệp vụ.
Mapper: chuyển đổi giữa các dạng.

Không nên trả thẳng JPA entity ra frontend nếu có thể tránh, vì:

- Lộ cấu trúc database.
- Dễ vòng lặp quan hệ.
- Khó kiểm soát response.
- Khó thay database schema.

Trả lời hội đồng:

Backend dùng kiến trúc multi-module theo domain. Mỗi module tách controller, use case, repository interface và infrastructure. Cách này giúp code rõ trách nhiệm, dễ test và dễ mở rộng hơn so với gom toàn bộ nghiệp vụ vào controller.

5. Database: Nguồn Sự Thật Của Hệ Thống

Database dùng:

PostgreSQL + pgvector

Database giữ:

- User, role.
- Publication, item/copy.
- Author, publisher, category, tag.
- Borrowing transaction.
- Reservation.
- Fine.
- Notification.
- Contact message/ticket.
- Wishlist, rating, search history.
- AI vector, metadata.
- Audit log, policy.

5.1. Publication Và Item

Cần phân biệt:

Publication = đầu sách/tác phẩm.
Item/Copy = bản sao vật lý cụ thể.

Ví dụ:

```
Publication: Clean Code.  
Item 1: mã vạch A, đang available.  
Item 2: mã vạch B, đang borrowed.  
Item 3: mã vạch C, đang maintenance.
```

Nếu code lại, đây là khái niệm cực quan trọng. Người dùng tìm publication, nhưng thủ thư mượn/trả item cụ thể.

5.2. Borrowing Transaction

Transaction biểu diễn một lần mượn/trả.

Nó liên quan:

```
User  
Item  
Borrow date  
Due date  
Return date  
Status  
Fine nếu có
```

5.3. Reservation

Reservation dùng khi user muốn giữ chỗ/đặt trước.

Logic thường cần kiểm:

- User có quyền đặt không?
- Publication có item available không?
- User đã đặt trước publication này chưa?
- Hàng chờ thứ mấy?
- Deadline nhận sách khi item sẵn sàng là gì?

5.4. Policy Trong Database

Project có circulation policy service:

```
CirculationPolicyService  
JdbcCirculationPolicyService
```

Ý tưởng:

Giới hạn mượn, ngày mượn, phí phạt, rule reservation không hardcode toàn bộ trong code.
Một phần cấu hình nằm trong DB để admin quản lý.

5.5. AI Schema Và pgvector

AI cần lưu:

```
chunk_text
embedding vector
publication_id
metadata
```

Semantic search sẽ:

```
Query text -> embedding -> so khoảng cách vector -> publication_ids liên quan
```

Trả lời hội đồng:

Database là nguồn sự thật của hệ thống. Điểm quan trọng là phân biệt publication và item: publication là đầu sách, item là bản sao cụ thể được mượn/trả. Ngoài dữ liệu thư viện truyền thống, database còn lưu vector embedding bằng pgvector để phục vụ semantic search.

6. Luồng Auth Và Phân Quyền

6.1. Các Chức Năng Auth

Auth gồm:

- Đăng ký.
- Verify email.
- Đăng nhập.
- Google OAuth2 callback.
- Refresh access token.
- Logout.
- Forgot password.
- Reset password.
- Onboarding profile.

Frontend liên quan:

```
AuthContext
authService.ts
LoginPage
RegisterPage
OAuth2CallbackPage
ForgotPasswordPage
ResetPasswordPage
```

Backend liên quan:

```
library-auth-module
AuthController
LoginUseCase
RefreshAccessTokenUseCase
VerifyEmailUseCase
ForgotPasswordUseCase
ResetPasswordUseCase
SecurityConfig
SecurityAspect
UserDetailsServiceImpl
```

6.2. Token Flow

Luồng đăng nhập:

```
User nhập email/password
-> frontend authService
-> backend /auth/login
-> backend xác thực
-> trả accessToken + refreshToken + role
-> frontend lưu localStorage
-> AuthContext cập nhật userType
```

Luồng gọi API:

```
axiosInstance đọc accessToken
-> gán Authorization Bearer
-> backend validate JWT
-> controller xử lý
```

Luồng token hết hạn:

```
Backend trả code 1403
-> axiosInstance gọi /auth/refresh-access-token bằng refreshToken
```

```
-> nhận token mới  
-> retry request cũ
```

6.3. Role-Based Access

Frontend chặn route bằng `ProtectedRoute`.

Backend vẫn phải chặn API bằng security.

Lý do:

```
Frontend route guard chỉ bảo vệ giao diện.  
Người dùng vẫn có thể gọi API bằng Postman/curl.  
Backend mới là lớp bảo mật thật.
```

Trả lời hội đồng:

Auth của project dùng access token và refresh token. Frontend lưu token, `axiosInstance` tự gắn Bearer token và refresh khi access token hết hạn. Route guard ở frontend giúp điều hướng trải nghiệm, nhưng phân quyền thật phải được backend kiểm tra theo role.

7. Luồng Catalog Và Search

7.1. Catalog Gồm Gì?

Catalog quản lý:

- Publication.
- Item/copy.
- Author.
- Publisher.
- Category.
- Tag.
- Cover/document.
- Public testimonials/system review.

Frontend:

```
publicationsService.ts  
categoriesService.ts  
authorsService.ts  
publishersService.ts  
tagsService.ts  
BookDetailPage  
SearchPage  
BookList  
BookDetails
```

```
CopyList  
CopyDetails
```

Backend:

```
library-catalog-module  
PublicationController  
ItemController  
CategoryController  
AuthorController  
PublisherController  
TagController  
PublicationRepositoryImpl  
SearchPublicationsUseCaseImpl  
CreatePublicationUseCaseImpl  
CreateItemUseCaseImpl
```

7.2. Search Thường Và Semantic Search

Search thường:

```
User nhập keyword  
-> frontend gọi publications/search  
-> backend query title/author/category/tag  
-> trả page result
```

Semantic search:

```
User nhập ý nghĩa cần tìm  
-> frontend gọi semanticSearchService  
-> backend hoặc AI endpoint  
-> AI tạo embedding query  
-> pgvector tìm chunks gần nhất  
-> trả publication_ids  
-> frontend/backend lấy publication detail
```

7.3. Khi Code Lại Search

Nên làm theo thứ tự:

1. Làm search thường bằng keyword.
2. Thêm filter category/type/author.
3. Thêm pagination/sort.
4. Thêm view grid/list.
5. Thêm search history.

6. Thêm semantic search.
7. Thêm recommendation/similar books.

Trả lời hội đồng:

Catalog là phần quản lý dữ liệu sách và bản sao. Search thường dựa vào dữ liệu có cấu trúc như title, author, category, còn semantic search dựa vào embedding vector để tìm theo ý nghĩa. Hai loại search bổ sung cho nhau.

8. Luồng Mượn Trả, Đặt Trước Và Phí Phạt

Đây là nghiệp vụ cốt lõi của thư viện.

Backend module:

```
library-circulation-module
```

Controller:

```
BorrowingTransactionController  
ReservationController  
FineController  
LibrarianController  
CirculationPolicyController  
TransactionNoteController
```

Use case:

```
BorrowRequestUseCase  
ReturnBookUseCase  
CreateReservationUseCase  
GetMyFinesUseCase  
PayFineUseCase  
PayAllFinesUseCase  
CirculationPolicyService
```

Frontend:

```
MyBooksPage  
ReservationsPage  
FinesPage  
Circulation  
Requests  
TransactionList  
reservationService.ts
```

```
transactionsService.ts  
fineService.ts  
circulationPolicyService.ts
```

8.1. Luồng Đặt Trước

```
User chọn publication  
-> bấm reserve  
-> backend kiểm user, publication, item availability, duplicate  
reservation  
-> tạo reservation  
-> gửi notification/mail nếu cần  
-> user thấy trạng thái reservation
```

8.2. Luồng Mượn Sách

```
User hoặc librarian tạo borrow request  
-> backend kiểm policy  
-> kiểm user có bị chặn không  
-> kiểm item available không  
-> tạo borrowing transaction  
-> đổi item status  
-> gửi notification/mail
```

8.3. Luồng Trả Sách

```
Librarian nhận sách  
-> backend tìm transaction đang borrowed  
-> set return date  
-> tính overdue nếu có  
-> tạo fine nếu trễ hạn  
-> đổi item status  
-> xử lý reservation kế tiếp nếu có  
-> gửi notification/mail
```

8.4. Luồng Phí Phạt

```
Transaction overdue  
-> fine được tạo  
-> user xem fine  
-> user thanh toán một phần/toàn bộ  
-> backend cập nhật trạng thái fine
```

8.5. Điểm Khó Của Circulation

Circulation khó vì có nhiều trạng thái:

- Item available/borrowed/reserved/lost/maintenance.
- Reservation pending/ready/cancelled/expired/fulfilled.
- Transaction requested/borrowed/returned/overdue.
- Fine unpaid/paid/waived.

Nếu code lại, cần vẽ state machine trước khi code.

Trả lời hội đồng:

Mượn trả là nghiệp vụ lỗi và phức tạp nhất vì phải đồng bộ trạng thái user, item, transaction, reservation và fine. Backend phải là nơi kiểm rule chính, frontend chỉ hiển thị và gửi thao tác.

9. Notification, Mail Và Contact Support

9.1. Notification

Notification dùng để báo trong hệ thống:

- Mượn sách thành công.
- Sắp đến hạn.
- Quá hạn.
- Reservation ready.
- Phí phạt.
- Contact ticket update.

Frontend:

```
NotificationContext
notificationService.ts
NotificationsPage
notificationLocalization.ts
```

Backend:

```
NotificationController
NotificationRepository
Kafka topic/event
```

9.2. Mail

Mail production dùng Zoho.

Luồng sender:

Trường hợp	Sender
Verify, reset password, mượn/trả/phạt/đặt trước	noreply@library74.uk
Thủ thư bấm Đã xử lý ở contact inbox	support@library74.uk

Backend dùng:

```
EmailService
EmailServiceImpl
```

Không hardcode password trong code. Password lấy từ `.env.prod`.

9.3. Contact Ticket

Contact có hai mặt:

```
User tạo ticket và xem ticket của mình.
Librarian/admin xem inbox và xử lý.
```

Frontend:

```
ContactPage
ContactTicketsPage
ContactInbox
contactService.ts
```

Backend:

```
ContactMessageController
```

Khi thủ thư xử lý:

```
Set trạng thái đã xử lý
Gửi mail hoàn tất hỗ trợ từ support alias
User thấy ticket đã resolved
```

Trả lời hội đồng:

Notification là thông báo trong hệ thống, còn email là kênh gửi ra ngoài. Project tách mail hệ thống `noreply` và mail hỗ trợ `support` để đúng ngữ cảnh giao tiếp. Contact ticket là luồng hỗ trợ người dùng, nối frontend user với inbox của librarian/admin.

10. Recommendation, Wishlist, Rating Và AI Gateway

Module:

```
library-recommendation-module
```

Chức năng:

- Wishlist.
- Rating.
- Search history.
- Recommendation.
- Similar publications.
- AI semantic search.
- AI callback.

Frontend:

```
wishlistService.ts  
recommendationService.ts  
semanticSearchService.ts  
searchHistoryService.ts  
WishlistPage  
SearchPage  
BookDetailPage
```

Backend:

```
WishlistController  
RatingController  
RecommendationController  
SimilarPublicationsController  
AiSearchController  
AiCallbackController  
AiGatewayService  
GetRecommendationsUseCaseImpl
```

10.1. Wishlist

Wishlist là hành vi user lưu sách quan tâm.

Nó cũng có thể là tín hiệu cho recommendation.

10.2. Rating

Rating cần rule:

User chỉ được rating publication đã mượn.
Mỗi user chỉ rating một lần cho một publication.

10.3. Search History

Search history giúp:

- User xem lại tìm kiếm.
- Hệ thống hiểu sở thích.
- AI/recommendation có thêm tín hiệu.

10.4. AI Gateway

Backend không nên biết chi tiết model AI.

Backend gọi AI qua:

AiGatewayService

Gateway chịu trách nhiệm:

- Gọi FastAPI.
- Parse response.
- Timeout.
- Fallback khi AI lỗi.

10.5. Fallback Là Bắt Buộc

AI không được làm hỏng nghiệp vụ lỗi.

Nếu AI lỗi:

Recommendation fallback về cached/trending.
Semantic search có thể trả empty hoặc fallback keyword.
Backend vẫn sống.

Trả lời hội đồng:

Recommendation module không chỉ có AI, mà còn có dữ liệu hành vi như wishlist, rating và search history. Backend gọi AI qua gateway để tách chi tiết AI service khỏi nghiệp vụ chính và có fallback khi AI không sẵn sàng.

11. AI Service: API, Worker Và ETL

AI nằm ở:

LMS_AI/

Các file chính:

```
api_service.py
worker.py
ai_etl/
ai_gateway/
scripts/
tests/
```

11.1. `api_service.py`

FastAPI service định nghĩa:

- `/health`.
- Semantic search.
- Similar publications.
- Recommendations.
- Process/vectorize/metadata endpoints.

Nó dùng:

```
SentenceTransformer
DatabaseConnectionPool
AIEtlPipeline
RecommenderEngine
```

11.2. `worker.py`

Worker dùng Celery và RabbitMQ.

Nhiệm vụ:

```
Nhận message process book.
Download PDF.
Chạy ETL pipeline.
Post callback về backend.
Ký callback bằng HMAC.
Retry khi lỗi tạm thời.
```

11.3. `ai_etl/`

ETL gồm:

```
pdf_processing.py: extract text, chunk.  
embedding_client.py: tạo embedding.  
llm_client.py: gọi Gemini sinh metadata.  
pipeline.py: orchestration ETL.  
db.py: thao tác database.  
tag_quality.py: kiểm chất lượng tag.
```

11.4. ai_gateway/

AI gateway phía Python chứa:

```
recommender.py  
config.py
```

Recommender kết hợp:

- Collaborative filtering.
- Content-based.
- Trending fallback.

11.5. Khi Code Lại AI

Nên code theo thứ tự:

1. `/health`.
2. Kết nối database.
3. Extract PDF text.
4. Chunk text.
5. Embedding một đoạn text.
6. Lưu vector.
7. Semantic search query vector.
8. Recommendation fallback đơn giản.
9. Thêm worker queue.
10. Thêm LLM metadata.
11. Thêm callback HMAC.

Trả lời hội đồng:

AI service được tách thành API và worker. API phục vụ request nhanh như semantic search và recommendation, còn worker xử lý tác vụ nặng như PDF ETL, embedding và metadata. Cách tách này giúp AI không làm nghẽn backend chính.

12. Deploy: Cách Hệ Thống Chạy Thật

File quan trọng:

```
docker-compose.prod.yml
deploy/Caddyfile
.env.prod
DEPLOY_RUNBOOK.md
```

12.1. Production Route

```
https://library74.uk
-> Caddy
-> frontend hoặc backend
```

Caddy route:

```
/api/* -> backend:8080
/ws/*  -> backend:8080
còn lại -> frontend:80
```

12.2. Environment

.env.prod chứa:

- Domain.
- Database password.
- Mail config.
- OAuth secret.
- JWT secret.
- AI callback secret.
- Public frontend variables.

Không commit **.env.prod**.

12.3. Rebuild Theo Phần

Sửa frontend:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml up -d --
build frontend
```

Sửa backend:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml up -d --  
build backend
```

Sửa AI:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml up -d --  
build ai-api ai-worker
```

12.4. Healthcheck

```
curl -I https://library74.uk  
docker exec lms_backend wget -qO- http://localhost:8080/actuator/health  
docker exec lms_ai_api curl -fsS http://localhost:8001/health
```

Trả lời hội đồng:

Production dùng Docker Compose để chạy toàn bộ stack. Caddy đứng trước xử lý HTTPS và route request. Khi sửa phần nào thì rebuild đúng service phần đó để giảm rủi ro. Sau deploy phải kiểm tra website, backend health và AI health.

13. Testing: Cách Giữ Dự Án Không Vỡ Khi Sửa

Các file test/tài liệu:

```
TEST_PLAN.md  
TEST_REPORT.md  
TEST_CASES.md  
learn/06_TESTING.md
```

13.1. FE Test

```
cd LMS_FE  
npm test -- --runInBand  
npm run build
```

Bao phủ:

- Error mapping.
- Language context.
- Theme context.

- Auth context.
- Upload context.
- Notification localization.
- Service contract.

13.2. BE Test

```
cd LMS_BE
mvn test
```

Bao phủ:

- Use case.
- Controller.
- AI gateway contract.
- Circulation.
- Auth.
- User.
- Recommendation.

13.3. AI Test

```
cd LMS_AI
pytest
```

Bao phủ:

- API contract.
- Worker contract.
- PDF processing.
- Config.
- Recommender.
- Pipeline.
- Database SQL contract.

13.4. Tư Duy Test Khi Code Lại

Mỗi lần thêm một chức năng, tự hỏi:

```
Logic nào phải unit test?
API contract nào phải test?
Database query nào dễ vỡ?
AI fallback nào cần test?
UI flow nào cần test bằng tay hoặc E2E?
```

Trả lời hội đồng:

Testing trong project không chỉ để có số lượng test, mà để bảo vệ các điểm dễ vỡ: logic dùng chung frontend, nghiệp vụ backend, contract BE-AI, native SQL database và AI fallback. Đây là lý do mỗi phần FE/BE/AI đều có bộ test riêng.

14. Nếu Code Lại Từ Đầu, Nên Làm Theo Thứ Tự Nào?

Đây là roadmap thực tế để dựng lại một phiên bản Library74.

Giai Đoạn 1: Skeleton

Làm tối thiểu:

```
React app.  
Spring Boot app.  
PostgreSQL.  
Docker Compose local.  
Health endpoints.  
Basic routing.
```

Mục tiêu:

```
Frontend gọi được backend.  
Backend gọi được database.  
Deploy local bằng compose.
```

Giai Đoạn 2: Auth Và Role

Làm:

```
User table.  
Role table.  
Register.  
Login.  
JWT access token.  
Refresh token.  
ProtectedRoute.  
Backend security.
```

Chưa cần Google OAuth ngay.

Giai Đoạn 3: Catalog

Làm:

```
Publication.  
Item/copy.  
Author.  
Category.  
Publisher.  
Search thường.  
Book detail.  
Librarian CRUD.
```

Đây là nền cho mọi nghiệp vụ sau.

Giai Đoạn 4: Circulation

Làm:

```
Borrow.  
Return.  
Reservation.  
Fine.  
Policy.  
My books.  
Fines page.  
Librarian circulation.
```

Trước khi code, vẽ state machine.

Giai Đoạn 5: Notification Và Mail

Làm:

```
Notification table.  
Notification API.  
Frontend notification page.  
Mail service.  
Verify email.  
Reset password.  
Borrow/return/fine mail.
```

Sau đó thêm Kafka nếu muốn bất đồng bộ tốt hơn.

Giai Đoạn 6: Contact Support

Làm:

```
Public contact form.  
User contact tickets.
```

```
Librarian/admin contact inbox.  
Resolve ticket.  
Support email.  
VI/EN text.
```

Giai Đoạn 7: Recommendation Cơ Bản

Làm:

```
Wishlist.  
Rating.  
Search history.  
Trending recommendation.  
Similar by category/tag.
```

Chưa cần AI phức tạp ngay.

Giai Đoạn 8: AI

Làm:

```
FastAPI /health.  
Semantic search endpoint.  
Embedding model.  
pgvector table.  
PDF extract/chunk/vector.  
Recommendation engine.  
Worker queue.  
LLM metadata.  
Callback HMAC.
```

Giai Đoạn 9: Admin Và Audit

Làm:

```
Admin user management.  
Librarian management.  
Policy management.  
Audit logs.  
Admin settings.
```

Giai Đoạn 10: Production

Làm:

```
docker-compose.prod.yml.  
Caddyfile.  
.env.prod.  
Backup.  
Healthcheck.  
Deploy runbook.  
Smoke test.
```

Trả lời hội đồng:

Nếu code lại, em không bắt đầu từ AI ngay. Em sẽ dựng skeleton, auth, catalog, circulation, notification/mail, contact, recommendation cơ bản, rồi mới thêm AI và production. Thứ tự này đúng vì AI và recommendation phụ thuộc vào dữ liệu sách, user và interaction có trước.

15. Cách Lăn Theo Một Chức Năng Trong Code

Khi muốn hiểu hoặc sửa một chức năng, đi theo 7 bước.

Bước 1: Tìm Page

Ví dụ contact ticket user:

```
LMS_FE/pages/user_pages/ContactTicketsPage.tsx
```

Bước 2: Tìm Service FE

```
LMS_FE/api/contactService.ts
```

Xem page gọi endpoint nào.

Bước 3: Tìm Controller BE

```
ContactMessageController
```

Xem endpoint nhận request ra sao.

Bước 4: Tìm Use Case/Service

Xem controller gọi service/use case nào.

Bước 5: Tìm Repository/Entity

Xem dữ liệu lưu bảng nào, field nào, trạng thái nào.

Bước 6: Tìm Side Effect

Side effect là hành động phụ:

```
Gửi mail.  
Tạo notification.  
Ghi audit log.  
Publish Kafka event.  
Gọi AI.
```

Bước 7: Tìm Test

```
*.test.ts  
*Test.java  
test_*.py
```

Nếu chưa có test, cần nhắc thêm test cho logic quan trọng.

Trả lời hội đồng:

Khi sửa một chức năng, em lần theo đường đi từ page frontend, service API, controller backend, use case, repository, database và các side effect như mail/notification/audit. Cách này giúp hiểu toàn bộ tác động của thay đổi.

16. Những Điểm Kỹ Thuật Cần Nhớ Chắc

16.1. Frontend

Cần nhớ:

- React component.
- Hook.
- Context.
- React Router.
- Lazy loading.
- Axios interceptor.
- Protected route.
- i18n tự quản bằng LanguageContext.
- Theme dark/light/system.
- Service API layer.

16.2. Backend

Cần nhớ:

- Spring Boot.

- Maven multi-module.
- Controller.
- DTO/request/response.
- Use case pattern.
- Repository interface.
- JPA entity.
- JDBC/native SQL.
- Spring Security.
- JWT/refresh token.
- Kafka event.
- Email service.
- Audit log.

16.3. Database

Cần nắm:

- PostgreSQL.
- ERD.
- Foreign key.
- Unique constraint.
- Transaction.
- Index.
- Migration.
- pgvector.
- Backup.

16.4. AI

Cần nắm:

- FastAPI.
- Pydantic model.
- Celery worker.
- RabbitMQ.
- PDF extraction.
- Chunking.
- Embedding.
- Vector similarity.
- Recommendation fallback.
- LLM metadata.
- HMAC callback.

16.5. Deploy

Cần nắm:

- Docker image/container.
- Docker Compose.

- Caddy reverse proxy.
- `.env.prod`.
- Volume.
- Healthcheck.
- Logs.
- Rollback.

16.6. Testing

Cần nắm:

- Unit test.
- Component test.
- Integration test.
- Contract test.
- E2E idea.
- Smoke test.
- Mock vs real dependency.

17. Các Luồng Quan Trọng Cần Vẽ Được

Nếu muốn thật sự nắm dự án, hãy tự vẽ lại các luồng này.

17.1. Login

```
LoginPage
-> authService.login
-> axiosInstance
-> AuthController
-> LoginUseCase
-> UserRepository
-> JWT/RefreshToken
-> localStorage/AuthContext
```

17.2. Search Sách

```
SearchPage
-> publicationsService hoặc semanticSearchService
-> PublicationController hoặc AiSearchController
-> SearchPublicationsUseCase hoặc AiGatewayService
-> PostgreSQL hoặc ai-api/pgvector
-> render grid/list
```

17.3. Mượn Sách

Circulation page

- > transactionsService
- > BorrowingTransactionController
- > BorrowRequestUseCase
- > CirculationPolicyService
- > Item/Transaction repository
- > Notification/Mail

17.4. Trả Sách Và Phạt

Librarian return action

- > ReturnBookUseCase
- > calculate overdue
- > create/update Fine
- > update Item status
- > update Transaction status
- > notify user

17.5. Contact Resolved Mail

ContactInbox

- > contactService resolve
- > ContactMessageController
- > update status resolved
- > EmailService sends from support@library74.uk
- > user receives support completion email

17.6. AI PDF Processing

Librarian uploads document

- > backend stores document URL
- > task to AI worker
- > worker downloads PDF
- > extract text
- > chunk
- > embedding
- > save vectors
- > callback backend with HMAC

18. Sai Lầm Dễ Mắc Khi Code Lại

18.1. Để Nghiệp Vụ Trong Frontend

Sai:

Frontend tự quyết user có được mượn sách không.

Đúng:

Frontend chỉ gửi request.
Backend kiểm rule.

18.2. Trộn Publication Và Item

Sai:

Mượn publication trực tiếp.

Đúng:

User tìm publication, nhưng transaction phải gắn với item/copy cụ thể.

18.3. Không Có Fallback AI

Sai:

AI lỗi thì trang chủ hoặc search chết.

Đúng:

AI lỗi thì fallback keyword/trending/cached, nghiệp vụ chính vẫn chạy.

18.4. Hardcode Secret

Sai:

Mật khẩu mail trong code.

Đúng:

Secret trong `.env.prod` hoặc secret manager.

18.5. Không Test Native SQL

Sai:

Đổi schema rồi chỉ compile Java.

Đúng:

Chạy integration test với PostgreSQL thật.

18.6. Deploy Toàn Bộ Khi Chỉ Sửa Một Page

Sai:

Sửa text frontend nhưng rebuild toàn bộ stack, tăng rủi ro.

Đúng:

Chỉ rebuild frontend nếu không đụng backend/AI/env.

19. Bài Tập Để Tự Code Lại Theo Từng Lát Cắt

Bài 1: Code Lại Auth Mini

Yêu cầu:

- Register.
- Login.
- JWT.
- Refresh token.
- Protected route.
- User role.

Kết quả cần đạt:

Student login vào `/userpage/dashboard`.
Librarian không vào admin route được.

Refresh token tự chạy khi access token hết hạn.

Bài 2: Code Lại Catalog Mini

Yêu cầu:

- Publication CRUD.
- Item CRUD.
- Search keyword.
- Book detail.

Điểm bắt buộc:

Publication và item phải là hai bảng/khái niệm khác nhau.

Bài 3: Code Lại Circulation Mini

Yêu cầu:

- Borrow item.
- Return item.
- Overdue fine.
- Reservation khi không còn item available.

Điểm bắt buộc:

Vẽ trạng thái item, transaction, reservation trước khi code.

Bài 4: Code Lại Contact Support

Yêu cầu:

- Public form.
- User ticket list.
- Librarian inbox.
- Resolve ticket.
- Send support mail.
- VI/EN UI.

Bài 5: Code Lại Semantic Search Mini

Yêu cầu:

- Extract text từ vài tài liệu mẫu.
- Chunk text.
- Tạo embedding.

- Lưu vector.
- Query semantic search.

Ban đầu có thể bỏ worker, chạy sync trước. Sau đó mới thêm queue.

Bài 6: Code Lại Deploy Mini

Yêu cầu:

- Dockerfile frontend.
- Dockerfile backend.
- docker-compose local.
- Caddy hoặc nginx reverse proxy.
- `.env`.
- Healthcheck.

20. Câu Hỏi Hội Đồng Hay Hỏi Ở Mức Tổng Hợp

Câu 1: Dự án của em có kiến trúc gì?

Dự án tách thành frontend React, backend Spring Boot multi-module, AI FastAPI/worker và PostgreSQL/pgvector. Backend được chia module theo domain như auth, user, catalog, circulation, recommendation và shared. Production chạy bằng Docker Compose với Caddy làm reverse proxy.

Câu 2: Vì sao backend không viết hết trong một module?

Vì nghiệp vụ lớn và nhiều domain. Tách module giúp mỗi phần có trách nhiệm rõ, dễ tìm code, dễ test và giảm phụ thuộc lộn xộn.

Câu 3: Vì sao frontend cần service API layer?

Vì service API layer tách logic gọi backend khỏi UI. Page chỉ quan tâm hiển thị và tương tác, còn service chịu trách nhiệm endpoint, payload và gọi `axiosInstance`.

Câu 4: Vì sao cần `axiosInstance`?

Vì mọi request đều cần xử lý chung như base URL, timeout, token, language, unwrap response, refresh token và lỗi chung. Đặt ở `axiosInstance` giúp tránh lặp code ở từng page.

Câu 5: Phần khó nhất của nghiệp vụ thư viện là gì?

Mượn trả và đặt trước khó nhất vì phải đồng bộ trạng thái user, item, transaction, reservation và fine. Nếu một trạng thái sai, dữ liệu thư viện sẽ sai.

Câu 6: AI trong dự án có vai trò gì?

AI hỗ trợ semantic search, recommendation và xử lý metadata từ PDF. AI không thay thế nghiệp vụ thư viện chính, mà bổ sung khả năng khám phá sách và gợi ý thông minh.

Câu 7: Nếu AI service lỗi thì hệ thống có chết không?

Không nên chết. Backend có gateway và fallback để recommendation có thể dùng cached/trending hoặc trả response hợp lệ. Nghiệp vụ chính như login, search thường, mượn trả vẫn phải chạy.

Câu 8: Làm sao biết deploy thành công?

Kiểm tra container `docker compose ps`, website <https://library74.uk>, backend actuator health, AI health và test chức năng vừa sửa bằng browser.

Câu 9: Làm sao tránh lỗi khi sửa code?

Lần theo luồng từ frontend page đến backend controller/use case/repository/database, kiểm side effect như mail/notification, sau đó chạy test liên quan. Với thay đổi lớn cần smoke test sau deploy.

Câu 10: Nếu phải code lại, em bắt đầu từ đâu?

Em sẽ bắt đầu từ skeleton frontend/backend/database, sau đó auth, catalog, circulation, notification/mail, contact, recommendation cơ bản, AI, admin/audit, rồi production deploy. Thứ tự này đi từ nền tảng đến chức năng nâng cao.

21. Tóm Tắt Một Phút

Library74 là hệ thống thư viện nhiều vai trò, được xây bằng React frontend, Spring Boot backend multi-module, FastAPI/Celery AI service và PostgreSQL/pgvector. Frontend tổ chức theo route, page, component, context và API service; `axiosInstance` xử lý token, language và refresh token. Backend chia domain thành auth, user, catalog, circulation, recommendation và shared; mỗi module đi theo luồng controller -> use case -> repository -> database. Database là nguồn sự thật, đặc biệt phải phân biệt publication và item. Circulation là nghiệp vụ khó nhất vì liên quan mượn, trả, đặt trước, phí phạt và policy. AI được tách riêng để hỗ trợ semantic search, recommendation và PDF ETL, có worker và fallback để không làm hỏng nghiệp vụ chính. Production chạy bằng Docker Compose với Caddy, còn testing bảo vệ các phần dễ vỡ như context frontend, use case backend, native SQL và contract BE-AI.

Nếu muốn code lại, hãy làm theo thứ tự: skeleton, auth, catalog, circulation, notification/mail, contact, recommendation cơ bản, AI, admin/audit, deploy và testing. Mỗi chức năng cần lần theo đường đi từ UI đến API, use case, database và side effect.

22. Mười Ý Chính Cần Nhớ

1. Library74 là hệ thống nhiều domain, không phải web CRUD đơn giản.
2. Frontend tách page, component, context, service API và utility.
3. `App.tsx` quản lý route, layout, protected route và lazy loading.
4. `axiosInstance` là cửa ra API, xử lý token, language và refresh token.
5. Backend là Spring Boot multi-module theo domain.
6. Luồng backend chuẩn là controller -> use case -> repository -> database.
7. Publication là đầu sách, item/copy là bản sao được mượn trả.
8. Circulation cần kiểm soát state của item, transaction, reservation và fine.
9. AI là service phụ trợ có API, worker, ETL, vector search và fallback.
10. Muốn sửa hoặc code lại một chức năng, hãy lần theo toàn bộ luồng từ UI đến database và test.